

# HOSPITAL RESOURCE MANAGEMENT

presnted by

T.Navadeep-CDS/2025/1852 |

B.Venkateswara Reddy-CDS/2025/1826

# PROJECT OVERVIEW

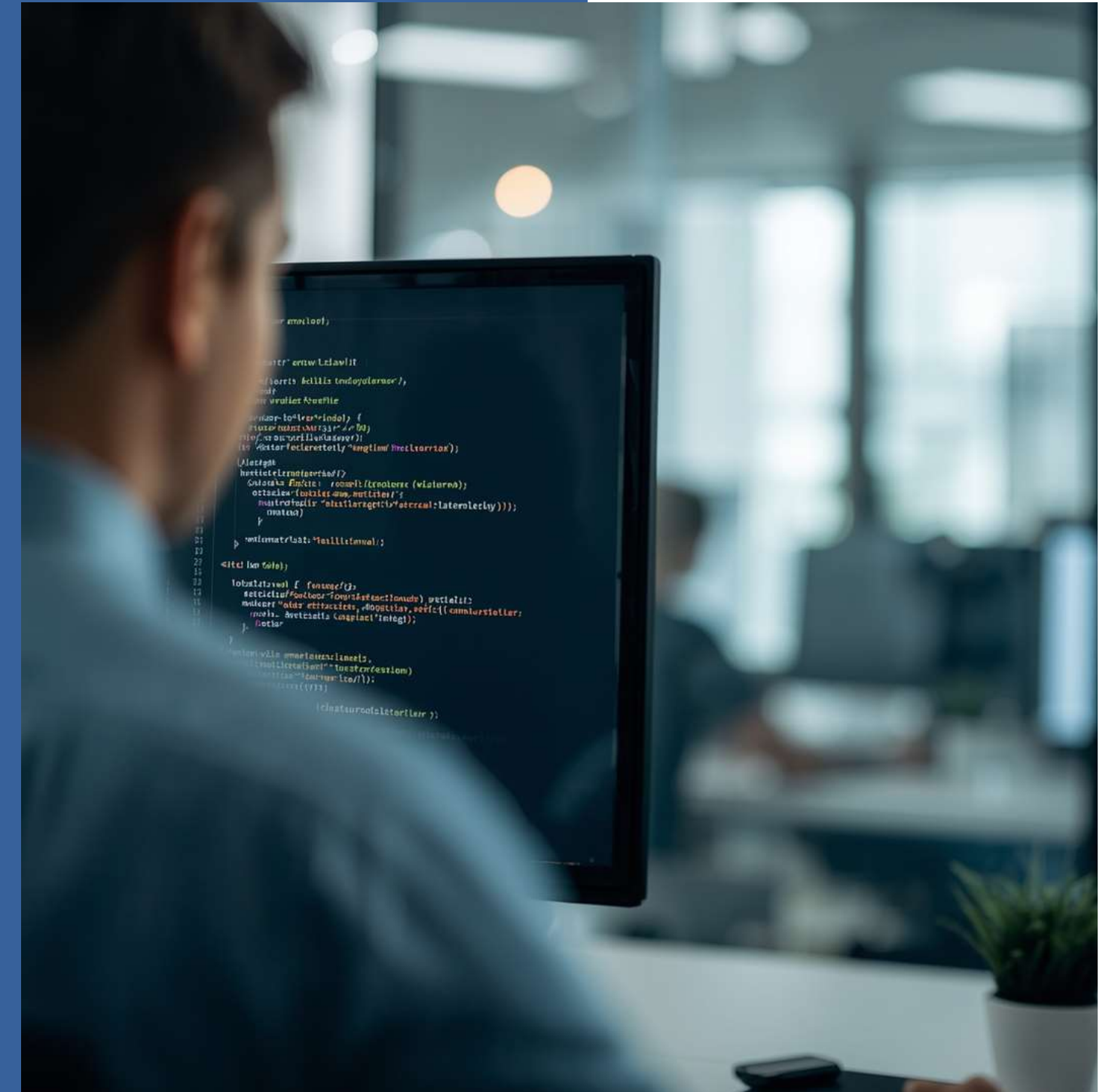
A menu-driven C application designed to manage hospital resources and simulate patient transfers between facilities. The system supports comprehensive CRUD operations—adding, updating, deleting, and searching hospital records—with emphasis on efficient data handling and real-world hospital coordination.



# PROJECT OBJECTIVES

Implement CRUD operations using C programming  
Practice dynamic memory allocation with malloc() and free()  
Use structured data types (struct) effectively

Build modular functions for maintainability and code reusability.  
Simulate hospital resource tracking and patient transfer basics  
to reflect real-world healthcare coordination scenarios.



# KEY FEATURES

## ADD & SEARCH

Add new hospital records dynamically with automatic memory allocation. Search hospitals instantly by unique ID for quick access.

## UPDATE

Update hospital details including name and available beds. Delete records safely with proper pointer adjustment and memory cleanup.

## DISPLA Y

Display complete hospital list with real-time data. Track bed availability and resource status across all registered hospitals.

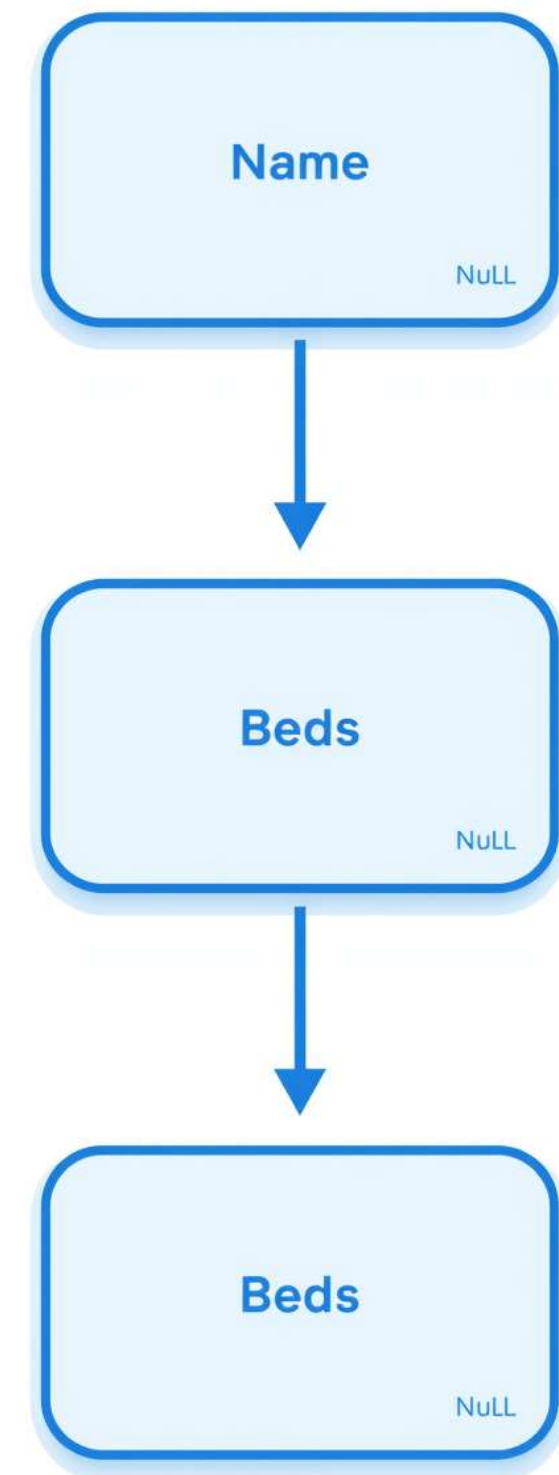
Our system leverages an efficient linked list data structure for flexible hospital record management, enabling dynamic operations with safe memory handling through proper malloc() and free() implementations.



# SYSTEM DESIGN & DATA STRUCTURE

The system uses a singly linked list to store hospital records dynamically. Each node in the list contains four key fields: Hospital ID (unique identifier), Hospital Name, Available Beds count, and a pointer to the next node in the sequence.

Dynamic memory allocation using malloc() enables flexible record management, allowing hospitals to be added or removed at runtime. This structure ensures efficient memory usage and supports scalable data handling without predefined size limits.





# CRUD OPERATIONS EXPLAINED

Create: Allocate memory using malloc(), input hospital data (ID, name, beds), insert new node at head of linked list. Read: Traverse the linked list sequentially, display all hospital records with their current data fields.

Update: Search by hospital ID, modify name and bed count fields when found. Delete: Locate target node, track previous pointer, adjust links to bypass node, free allocated memory. Search: Linear traversal comparing IDs, return record or not found.



# ALGORITHM WORKFLOW

Add Node: create, input, link to head, update head pointer

Delete Node: search node, track previous, update links, free memory

Search Node: traverse list, compare IDs, return result

Update Node: locate by ID, overwrite details as needed

Each algorithm follows a systematic approach to maintain data integrity. Adding inserts at the head for  $O(1)$  efficiency. Deletion carefully manages pointers before freeing memory. Search performs linear traversal until match found. Update combines search with field modification.



# TECHNOLOGIES USED

## C LANGUAGE & GCC COMPILER

Dynamic memory allocation using malloc() and free() for flexible data handling. Pointer manipulation for linked list traversal and node management.

## LINKED LISTS

Singly linked list data structure for efficient hospital record storage. Sequential access patterns optimized for CRUD operations.

## MODULAR DESIGN

Modular programming approach with separate functions for each operation. Clean code organization for maintainability and debugging.

This project is built entirely in C without external libraries or databases, demonstrating core programming fundamentals and efficient resource management through clean, modular code architecture.



# HOW TO RUN THE PROGRAM

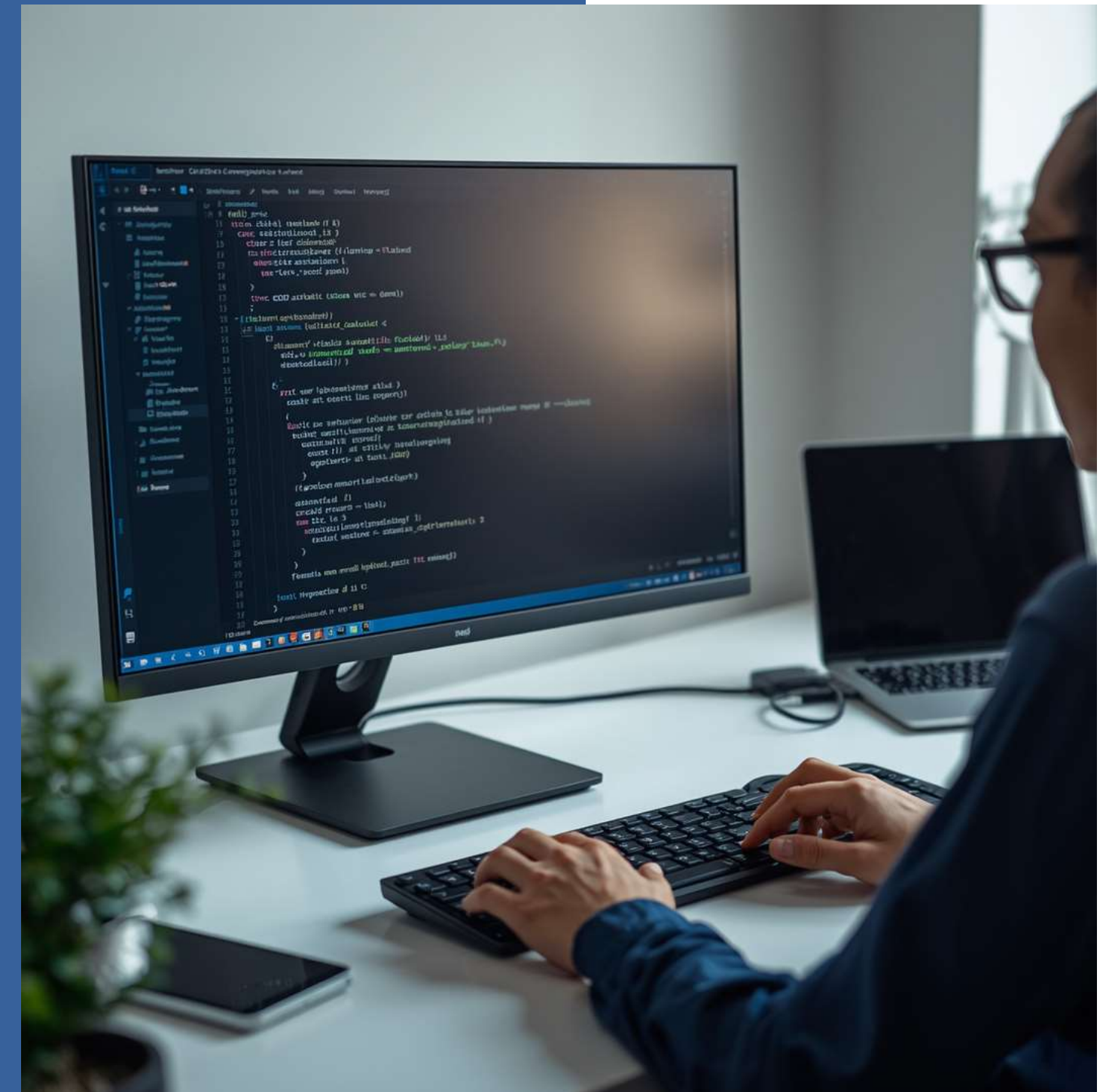
Step 1: Compile using `gcc hospital_graph.c -o hospital`

Step 2: Execute with `./hospital`

Menu interface guides all operations

Menu Options Available:

- Add – Insert new hospital records
- Delete – Remove existing records
- Update – Modify hospital details
- Search – Find hospitals by ID
- Display – View all records
- Exit – Close the program



# LIMITATIONS

## DATA STRUCTURE CONSTRAINT

Uses linked list instead of full graph representation, limiting complex hospital network modeling and multi-path transfer scenarios.

## BASIC SIMULATION

No persistent file storage implemented. All hospital data is lost when the program exits, requiring re-entry each session.

## NO PERSISTENCE

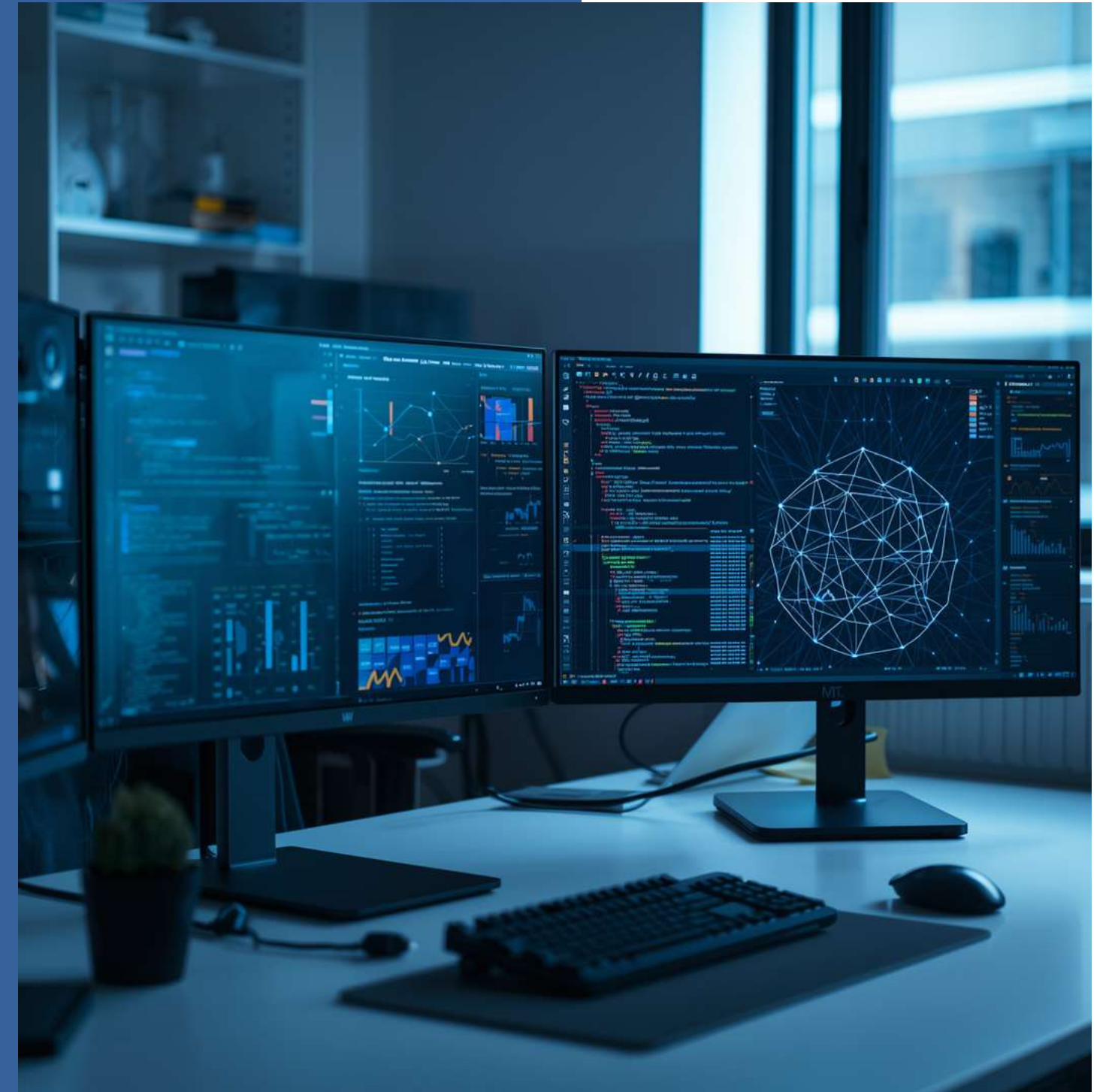
Basic patient transfer simulation only. No real-time hospital connectivity or network communication features.

While functional for demonstration purposes, this system has several constraints that limit its real-world applicability. These limitations represent opportunities for future development and enhancement of the hospital resource management system.

# FUTURE ENHANCEMENT

**S** Convert to graph structure (adjacency list)  
Implement Dijkstra's algorithm for optimization  
Add file handling for persistent storage  
Develop GUI with C++ or Python

These planned improvements will transform the current linked list implementation into a full graph-based system, enabling realistic patient transfer simulations with shortest path calculations. File storage ensures data persistence, while a graphical interface and authentication features will enhance usability and security.





The background is a solid medium blue. In the top-left corner, there is a large white circle with a thick dark blue border. In the bottom-right corner, there is a large dark blue circle with a thick white border. A thin white line forms a square in the top-right corner. Another thin white line forms a square in the bottom-left corner.

**THANK  
YOU!**